ASSIGNMENT 4

Deep Learning for NLP

Sr. No.	Name of Student	PRN	CO Mapped
1	Dhawse Spandan Yashwant	23070122085	CO5
2	Garvit Tyagi	23070122094	CO5

CA3 Project Title	RedactIQ: Explainable PII Detection and Redaction with Confidence-Scored Audit Trails

Aim

To implement a bidirectional Long Short-Term Memory network that labels every token of a sentence with a BIO tag, so that personally identifiable information is detected by a learned sequence model rather than by hand-written rules.

The model forms the neural detector of the CA3 project RedactIQ and is evaluated on entities that never appear in the training data.

Learning Outcome:

After completing this experiment, students will be able to:

- Convert character-level entity spans into a BIO tagging scheme suitable for a neural sequence labeller, and decode the predicted tags back into spans.
- Implement and train a bidirectional LSTM token classifier in PyTorch, including padding, masking of the loss and regularisation.
- Evaluate a sequence labeller at entity level and analyse, through an ablation study, which model component is responsible for its performance.

Theory

A Recurrent Neural Network processes a sequence one step at a time and carries a hidden state from each step to the next, so the representation of a word depends on the words before it. Training uses backpropagation through time, and because the gradient is multiplied by the same weight matrix at every step, it shrinks towards zero over long sequences. This is the vanishing gradient problem, and it prevents a plain RNN from learning long-range dependencies.

The Long Short-Term Memory network solves this with a separate cell state and three gates. The forget gate decides what to discard from the cell state, the input gate decides what new information to add, and the output gate decides what to expose as the hidden state. Because the cell state is updated additively rather than multiplicatively, the gradient can flow across many steps. The Gated Recurrent Unit is a lighter variant with only an update gate and a reset gate and no separate cell state, which gives fewer parameters and faster training at broadly similar accuracy.

A single LSTM reads left to right, so when it labels a word it has not yet seen the words that follow. Entity recognition needs both sides: in the fragment "forward the letter to X before Friday" the decision about X depends on the words after it. A bidirectional LSTM therefore runs one LSTM forward and another backward and concatenates their hidden states, so every token is represented by its full left and right context.

Entities can span several tokens, so the labels use the BIO scheme: B- marks the first token of an entity, I- marks a continuation, and O marks a token outside any entity. This makes it possible to represent a two-word name as one entity and to separate two entities that are adjacent. Sentences are padded to a fixed length, and the padded positions are given the label -100 so that the cross-entropy loss ignores them and the model is not trained on padding.

Two design choices matter more than the network itself. First, identifier strings are almost all unique, so they fall below the vocabulary threshold and become the unknown token; a word shape feature, which records whether a token is Title-case, all digits, contains an at sign, or mixes capitals and digits, lets the model recognise an unseen name or identifier from its form. Second, if training always shows the real word, the model memorises the training names and predicts O for every unseen one. Word dropout, which randomly replaces a fraction of the input words by the unknown token during training, forces the model to rely on shape and context and is what makes it generalise.

The model is evaluated at entity level rather than token level: a prediction counts as correct only when the decoded span and its label both match the gold entity exactly. Precision, recall and F1 are reported per entity type, and an ablation study removes one component at a time to show how much each contributes.

Algorithm

1. Import PyTorch and load the spaCy model used for tokenisation.
2. Generate a training corpus and a test corpus from sentence templates, recording the character span and label of every PII item. Use disjoint sets of names, organisations and distractor phrases for the two corpora, so that no test entity is ever seen during training.
3. Include adversarial distractor templates in which a Title-case phrase occupies an entity-like position but is not PII, so that word shape alone cannot solve the task.
4. Convert each sentence and its spans into a sequence of BIO tags by checking, for every token, whether it falls inside a gold span and whether it is the first token of that span.
5. Build the word vocabulary from tokens that occur at least three times, and define eight word shape classes covering e-mail, ten-digit number, other digits, Title-case, all-capitals, PAN-like alphanumeric, lowercase and other.
6. Encode every sentence as a word index sequence, a shape index sequence and a tag index sequence, padded to a fixed length with the label -100 at the padded positions.
7. Define the network: a word embedding, a shape embedding, their concatenation fed to a bidirectional LSTM, dropout, and a linear layer projecting the concatenated forward and backward states onto the tag set.
8. Train with the Adam optimiser and a cross-entropy loss that ignores the index -100, applying word dropout to each batch so that a fraction of the input words is replaced by the unknown token.
9. Decode the predicted tag sequences back into entity spans, and compute entity-level TP, FP and FN per entity type together with precision, recall and F1.
10. Repeat the training and evaluation with the shape features removed and with word dropout removed, to measure the contribution of each, and finally tag one completely unseen sentence.

Program

```python
import random
import torch
import torch.nn as nn
import spacy

nlp = spacy.load("en_core_web_sm")
random.seed(42)
torch.manual_seed(42)

# ---------------- Step 1 : Corpus with exact PII spans ----------------
# The test names and organisations are DISJOINT from the training ones, so the
# model is forced to generalise instead of memorising entity strings.
NAMES_TR = ["Rohan Mehta", "Anita Desai", "Kabir Rao", "Farhan Qureshi",
            "Priya Sharma", "Neha Kulkarni"]
NAMES_TE = ["Meera Joshi", "Vikram Nair", "Ayesha Siddiqui", "Dev Malhotra"]
ORGS_TR = ["Infosys Limited", "Wipro Technologies", "Symbiosis Institute"]
ORGS_TE = ["Persistent Systems", "Tata Motors"]
# Title-case phrases that are NOT PII. These are disjoint too, so at test time
# the model meets unseen Title-case pairs that look exactly like a person name.
DIST_TR = ["Registrar Office", "Human Resources", "Finance Department"]
DIST_TE = ["Placement Cell", "Academic Council", "Transport Section"]

def pan():
    return "".join(random.choices("ABCDEFGHIJKLMNOPQRSTUVWXYZ", k=5)) + \
           "".join(random.choices("0123456789", k=4)) + random.choice("ABCDEFGH")

def phone():
    return str(random.randint(6, 9)) + "".join(random.choices("0123456789", k=9))

def mail(n):
    return n.lower().replace(" ", ".") + "@sitpune.edu.in"

def build(parts):
    """parts = [(chunk, label or 0), ...] -> sentence text and labelled spans."""
    text, spans = "", []
    for chunk, lab in parts:
        if lab:
            spans.append((len(text), len(text) + len(chunk), lab))
        text += chunk
    return text, spans
```

```python
# (n)ame, (o)rganisation, (d)istractor are substituted into each template.
TEMPLATES = [
    lambda n, o, d: build([("Please contact ", 0), (n, "PERSON"), (" at ", 0),
                           (mail(n), "EMAIL"), (" or on ", 0), (phone(), "PHONE"), (".", 0)]),
    lambda n, o, d: build([("Employee ", 0), (n, "PERSON"), (" of ", 0), (o, "ORG"),
                           (" has PAN ", 0), (pan(), "PAN"), (".", 0)]),
    lambda n, o, d: build([("Ticket raised by ", 0), (n, "PERSON"),
                           (". Registered mobile is ", 0), (phone(), "PHONE"), (".", 0)]),
    lambda n, o, d: build([("The applicant ", 0), (n, "PERSON"), (" submitted PAN ", 0),
                           (pan(), "PAN"), (" for verification.", 0)]),
    lambda n, o, d: build([("Our records show that ", 0), (n, "PERSON"),
                           (" works with ", 0), (o, "ORG"), (".", 0)]),
    lambda n, o, d: build([("The compliance review meeting is scheduled next week.", 0)]),
    # Adversarial distractors: Title-case phrases in entity-like positions that
    # are NOT PII, so word shape alone cannot solve the task.
    lambda n, o, d: build([("Kindly forward the letter to the ", 0), (d, 0),
                           (" before Friday.", 0)]),
    lambda n, o, d: build([("Please contact the ", 0), (d, 0), (" team on Monday.", 0)]),
    lambda n, o, d: build([("The ", 0), (d, 0), (" will review the request in March.", 0)]),
]

def make(n, names, orgs, dists):
    return [random.choice(TEMPLATES)(random.choice(names), random.choice(orgs),
                                     random.choice(dists)) for _ in range(n)]

train_raw = make(900, NAMES_TR, ORGS_TR, DIST_TR)
test_raw = make(250, NAMES_TE, ORGS_TE, DIST_TE)
print("1. CORPUS :", len(train_raw), "train /", len(test_raw), "test sentences")
print("   train and test use DISJOINT names and organisations")
print("   sample :", train_raw[0][0])

# ---------------- Step 2 : Convert character spans to BIO tags ----------------
LABELS = ["PERSON", "ORG", "EMAIL", "PHONE", "PAN"]
TAGS = ["O"] + [p + "-" + l for l in LABELS for p in ("B", "I")]
tag2i = {t: i for i, t in enumerate(TAGS)}

def to_bio(text, spans):
    """B- marks the first token of an entity, I- the rest, O everything else."""
    words, tags = [], []
    for t in nlp(text):
        if t.is_space:
            continue
        s, e = t.idx, t.idx + len(t.text)
        tag = "O"
        for a, b, lab in spans:
            if a <= s and e <= b:
                tag = ("B-" if s == a else "I-") + lab
                break
        words.append(t.text)
        tags.append(tag)
    return words, tags

train = [to_bio(t, s) for t, s in train_raw]
test = [to_bio(t, s) for t, s in test_raw]
print("\n2. BIO TAGGING :", len(TAGS), "tag types")
for w, t in list(zip(*train[0]))[:6]:
    print(f"   {w:<24} {t}")
```

```python
# ---------------- Step 3 : Word shape features ----------------
# Entity strings are rare, so most become <unk>. A shape feature lets the model
# recognise an unseen name or identifier from its FORM rather than its identity.
def shape(w):
    if "@" in w:                                    return 1   # e-mail
    if w.isdigit():                                 return 2 if len(w) == 10 else 3
    if w.isalpha() and w.istitle():                 return 4   # Titlecase word
    if w.isalpha() and w.isupper():                 return 5   # ALLCAPS
    if w.isalnum() and w.isupper() and any(c.isdigit() for c in w):
                                                    return 6   # PAN-like AAAAA9999A
    if w.isalpha():                                 return 7   # lowercase word
    return 0

PAD, UNK, MAXLEN = 0, 1, 24
counts = {}
for ws, _ in train:
    for w in ws:
        counts[w.lower()] = counts.get(w.lower(), 0) + 1
vocab = {"<pad>": PAD, "<unk>": UNK}
for w, c in counts.items():
    if c >= 3:
        vocab[w] = len(vocab)

def encode(words, tags):
    x = [vocab.get(w.lower(), UNK) for w in words][:MAXLEN]
    s = [shape(w) for w in words][:MAXLEN]
    y = [tag2i[t] for t in tags][:MAXLEN]
    p = MAXLEN - len(x)
    return x + [PAD] * p, s + [0] * p, y + [-100] * p    # -100 is ignored by the loss

def tensors(rows):
    a, b, c = zip(*[encode(w, t) for w, t in rows])
    return torch.tensor(a), torch.tensor(b), torch.tensor(c)

Xtr, Str, Ytr = tensors(train)
Xte, Ste, Yte = tensors(test)
oov = sum(1 for ws, _ in test for w in ws if w.lower() not in vocab)
print(f"\n3. ENCODING : vocabulary = {len(vocab)} words, 8 shape classes")
print(f"   {oov} test tokens are out of vocabulary and rely on shape alone")

# ---------------- Step 4 : BiLSTM token classifier ----------------
```

```python
class BiLSTMTagger(nn.Module):
    def __init__(self, nvocab, ntags, emb=64, shp=16, hid=96, use_shape=True):
        super().__init__()
        self.use_shape = use_shape
        self.emb = nn.Embedding(nvocab, emb, padding_idx=PAD)
        self.shape_emb = nn.Embedding(8, shp)
        self.lstm = nn.LSTM(emb + (shp if use_shape else 0), hid,
                            bidirectional=True, batch_first=True)
        self.drop = nn.Dropout(0.3)
        self.fc = nn.Linear(hid * 2, ntags)      # hid*2 because it is bidirectional

    def forward(self, x, s):
        z = self.emb(x)
        if self.use_shape:
            z = torch.cat([z, self.shape_emb(s)], dim=-1)
        h, _ = self.lstm(z)
        return self.fc(self.drop(h))

def word_dropout(x, p=0.4):
    """Randomly replace real words by <unk> during training. Without this the
    model memorises the training names and predicts O for every unseen one."""
    m = (torch.rand_like(x, dtype=torch.float) < p) & (x != PAD)
    return torch.where(m, torch.full_like(x, UNK), x)

def train_model(use_shape, use_drop, epochs=25, quiet=True):
    torch.manual_seed(42)
    m = BiLSTMTagger(len(vocab), len(TAGS), use_shape=use_shape)
    opt = torch.optim.Adam(m.parameters(), lr=2e-3)
    lossf = nn.CrossEntropyLoss(ignore_index=-100)   # padding must not train
    for ep in range(1, epochs + 1):
        m.train()
        tot = 0.0
        for i in range(0, len(Xtr), 32):
            xb, sb, yb = Xtr[i:i+32], Str[i:i+32], Ytr[i:i+32]
            opt.zero_grad()
            loss = lossf(m(word_dropout(xb) if use_drop else xb, sb)
                         .reshape(-1, len(TAGS)), yb.reshape(-1))
            loss.backward()
            opt.step()
            tot += loss.item() * len(xb)
        if not quiet and ep in (1, 5, 15, 25):
            print(f"   epoch {ep:>2}  training loss = {tot/len(Xtr):.4f}")
    return m

print(f"\n4. MODEL : BiLSTM, "
      f"{sum(p.numel() for p in BiLSTMTagger(len(vocab), len(TAGS)).parameters()):,}"
      f" parameters")
model = train_model(True, True, quiet=False)

# ---------------- Step 5 : Entity-level evaluation ----------------
```

```python
def spans_from_bio(tags):
    """Decode a BIO tag sequence back into (start, end, label) entity spans."""
    out, cur = [], None
    for i, t in enumerate(tags):
        if t.startswith("B-"):
            if cur: out.append(cur)
            cur = [i, i + 1, t[2:]]
        elif t.startswith("I-") and cur and cur[2] == t[2:]:
            cur[1] = i + 1
        else:
            if cur: out.append(cur)
            cur = None
    if cur: out.append(cur)
    return [tuple(s) for s in out]

def evaluate(m):
    m.eval()
    with torch.no_grad():
        pred = m(Xte, Ste).argmax(-1)
    tp = fp = fn = 0
    per = {}
    for r in range(len(Xte)):
        keep = Yte[r] != -100
        g = spans_from_bio([TAGS[i] for i in Yte[r][keep].tolist()])
        p_ = spans_from_bio([TAGS[i] for i in pred[r][keep].tolist()])
        for s in p_:
            if s in g: tp += 1; per.setdefault(s[2], [0, 0, 0])[0] += 1
            else:      fp += 1; per.setdefault(s[2], [0, 0, 0])[1] += 1
        for s in g:
            if s not in p_: fn += 1; per.setdefault(s[2], [0, 0, 0])[2] += 1
    P = tp / (tp + fp) if tp + fp else 0.0
    R = tp / (tp + fn) if tp + fn else 0.0
    return tp, fp, fn, P, R, (2*P*R/(P+R) if P+R else 0.0), per

tp, fp, fn, P, R, F, per = evaluate(model)
print(f"\n5. ENTITY-LEVEL EVALUATION ON UNSEEN ENTITIES")
print(f"   {'Entity':<10}{'TP':<6}{'FP':<6}{'FN':<6}{'Prec':<9}{'Recall':<9}F1")
for lab in sorted(per):
    a, b, c = per[lab]
    p_ = a / (a + b) if a + b else 0.0
    r_ = a / (a + c) if a + c else 0.0
    print(f"   {lab:<10}{a:<6}{b:<6}{c:<6}{p_:<9.3f}{r_:<9.3f}"
          f"{2*p_*r_/(p_+r_) if p_+r_ else 0:.3f}")
print(f"   {'OVERALL':<10}{tp:<6}{fp:<6}{fn:<6}{P:<9.3f}{R:<9.3f}{F:.3f}")

# ---------------- Ablation : which component does the work? ----------------
print(f"\n   ABLATION STUDY")
print(f"   {'Configuration':<34}{'Prec':<9}{'Recall':<9}F1")
for name, sh, dr in [("word embedding only", False, False),
                     ("+ word shape features", True, False),
                     ("+ shape + word dropout", True, True)]:
    _, _, _, p2, r2, f2, _ = evaluate(train_model(sh, dr))
    print(f"   {name:<34}{p2:<9.3f}{r2:<9.3f}{f2:.3f}")
```

```python
# ---------------- Step 6 : Tagging a completely unseen sentence ----------------
demo = "Please contact Meera Joshi at meera.joshi@sitpune.edu.in or on 9871234560."
words = [t.text for t in nlp(demo) if not t.is_space]
x, s, _ = encode(words, ["O"] * len(words))
with torch.no_grad():
    out = model(torch.tensor([x]), torch.tensor([s])).argmax(-1)[0][:len(words)]
print("\n6. PREDICTION ON A SENTENCE THE MODEL HAS NEVER SEEN")
for w, i in zip(words, out.tolist()):
    if TAGS[i] != "O":
        print(f"   {w:<30} {TAGS[i]:<10} (in vocabulary: "
              f"{'yes' if w.lower() in vocab else 'no, shape only'})")
```

Output

```
1. CORPUS : 900 train / 250 test sentences
   train and test use DISJOINT names and organisations
   sample : Employee Rohan Mehta of Symbiosis Institute has PAN GDCTO5002A.

2. BIO TAGGING : 11 tag types
   Employee                O
   Rohan                   B-PERSON
   Mehta                   I-PERSON
   of                      O
   Symbiosis               B-ORG
   Institute               I-ORG
```

```
3. ENCODING : vocabulary = 78 words, 8 shape classes
   700 test tokens are out of vocabulary and rely on shape alone

4. MODEL : BiLSTM, 143,947 parameters
   epoch  1  training loss = 1.0271
   epoch  5  training loss = 0.0123
   epoch 15  training loss = 0.0010
   epoch 25  training loss = 0.0004
```

```
5. ENTITY-LEVEL EVALUATION ON UNSEEN ENTITIES
   Entity    TP    FP    FN    Prec     Recall   F1
   EMAIL     36    0     0     1.000    1.000    1.000
   ORG       46    0     0     1.000    1.000    1.000
   PAN       20    0     0     1.000    1.000    1.000
   PERSON    135   0     0     1.000    1.000    1.000
   PHONE     69    0     0     1.000    1.000    1.000
   OVERALL   306   0     0     1.000    1.000    1.000
```

```
   ABLATION STUDY
   Configuration                    Prec     Recall   F1
   word embedding only              0.183    0.291    0.224
   + word shape features            0.290    0.353    0.319
   + shape + word dropout           1.000    1.000    1.000
```

```
6. PREDICTION ON A SENTENCE THE MODEL HAS NEVER SEEN
   Meera                        B-PERSON   (in vocabulary: no, shape only)
   Joshi                        I-PERSON   (in vocabulary: no, shape only)
   meera.joshi@sitpune.edu.in   B-EMAIL    (in vocabulary: no, shape only)
   9871234560                   B-PHONE    (in vocabulary: no, shape only)
```

Explanation of the Output

The BIO conversion is visible in the first block: Rohan receives B-PERSON and Mehta receives I-PERSON, so the two tokens form a single entity rather than two. The vocabulary contains only 78 words even though the corpus has 900 sentences, because every name, e-mail address and identifier is unique and falls below the frequency threshold. As a result 700 test tokens are out of vocabulary and can only be recognised from their shape and their context.

The complete model reaches precision, recall and F1 of 1.000 on entities that never occur in training. This score should be read together with the ablation table rather than on its own: the corpus is generated from a fixed set of templates, so once the model has learned the context patterns the task is fully

solvable. What the result does establish is generalisation to unseen entities, since every test name, organisation and distractor phrase is new.

The ablation study is the substantive result. A plain word-embedding BiLSTM reaches an F1 of only 0.224 on the same data, because the entity tokens are all unknown and the model has memorised the training names. Adding word shape features raises this to 0.319, which recovers the structured identifiers whose form is distinctive but not the names. Adding word dropout raises it to 1.000, because the model can no longer rely on word identity during training and is forced to use shape and context, which is exactly what transfers to unseen entities.

The final block confirms this behaviour on a sentence written from scratch. Meera and Joshi are tagged B-PERSON and I-PERSON, the e-mail address and the telephone number are tagged correctly, and the report notes that none of these tokens is in the vocabulary, so all four decisions rest on shape and context alone.

Conclusion

In this experiment a bidirectional LSTM was implemented in PyTorch to detect personally identifiable information as a sequence labelling task. Character-level entity spans were converted into BIO tags, sentences were padded with the loss masked at the padded positions, and the predicted tags were decoded back into entity spans for evaluation. The model was tested on names, organisations and distractor phrases that were entirely disjoint from the training data, and it reached an entity-level F1 of 1.000 on that unseen material. The ablation study showed that the network alone is not responsible for this: a plain word-embedding BiLSTM achieves only 0.224, adding word shape features raises it to 0.319, and adding word dropout raises it to 1.000. The explanation is that identifier strings are almost always out of vocabulary, so the model must learn from the form of a token and from its surrounding words rather than from its identity, and word dropout is what forces it to do so. The perfect final score reflects a templated synthetic corpus in which the contexts are highly regular, and this is stated as a limitation rather than as a general accuracy claim. The trained tagger is added as the third detector of RedactIQ, alongside the regular-expression and spaCy detectors of Assignment 2.